

Bridge OffGas

Documentazione tecnica e operativa

Progetto OffGas

Aprile 2026

Indice

1. Scopo del documento	2
2. Ruolo del bridge nel sistema OffGas	2
3. Posizionamento del bridge nell'architettura	2
4. Comunicazioni gestite dal bridge	2
4.1 Comunicazione Bluetooth con Arduino	2
4.2 Comunicazione MQTT con Node-RED	3
5. Struttura del file bridge.py	3
5.1 Classe Config	3
5.2 Classe BluetoothManager	3
5.3 Classe FanController	4
5.4 Classe MQTTManager	4
5.5 Classe Bridge	4
6. Acquisizione del dato dal sensore	4
6.1 Lettura dalla seriale	4
6.2 Validazione e parsing	4
7. Pubblicazione della telemetria	5
7.1 Significato dei campi	5
8. Pubblicazione degli alert	5
9. Ricezione dei comandi da Node-RED	6
9.1 Comandi automatici JSON	6
9.2 Comandi manuali	6
10. Gestione della ventola	6
11. Configurazione MQTT e avvio operativo	6
12. Cosa il bridge non fa	7
13. Vantaggi della struttura attuale	7
14. Riepilogo finale	7

1. Scopo del documento

Questo documento descrive il ruolo del **Bridge Python** nel progetto OffGas nella sua forma attuale.

L'obiettivo è spiegare:

- quali responsabilità appartengono al bridge;
- come il bridge comunica con Arduino, Node-RED e il broker MQTT;
- come è organizzato il codice a classi;
- quali dati vengono pubblicati e quali comandi vengono ricevuti;
- quali sono i limiti intenzionali del bridge all'interno dell'architettura complessiva.

Il bridge viene trattato come un **gateway edge** tra il prototipo fisico e l'infrastruttura software del sistema.

2. Ruolo del bridge nel sistema OffGas

Nel sistema OffGas il bridge ha un compito preciso e delimitato: fare da collegamento tra il prototipo reale **G1** e la logica cloud implementata in **Node-RED**.

Il bridge **non** implementa la logica di analisi del sistema e **non** calcola la soglia di sicurezza o la predizione. Queste elaborazioni appartengono a Node-RED.

Le responsabilità del bridge sono quindi quattro:

1. leggere il valore di gas inviato da Arduino tramite Bluetooth;
2. pubblicare la telemetria su MQTT;
3. ricevere da Node-RED i comandi automatici o manuali;
4. inoltrare ad Arduino i comandi di controllo della ventola.

Questa separazione consente di mantenere il bridge semplice, riutilizzabile e focalizzato sulla comunicazione hardware.

3. Posizionamento del bridge nell'architettura

Il flusso generale del sistema è il seguente:

Arduino -> Bridge -> MQTT Broker -> Node-RED -> MQTT Broker -> Bridge -> Arduino

In dettaglio:

- Arduino legge il sensore MQ-2 e invia il valore al bridge tramite il modulo **HC-05**;
- il bridge interpreta il dato e lo pubblica sul topic MQTT della telemetria;
- Node-RED riceve il dato, aggiorna lo stato del sistema, esegue prediction e anomaly detection;
- Node-RED pubblica un comando sul topic dei comandi;
- il bridge riceve il comando e attiva o spegne la ventola tramite Arduino.

Il bridge è quindi un componente intermedio: non prende decisioni di alto livello, ma rende possibile il collegamento tra dispositivo fisico e logica di orchestrazione.

4. Comunicazioni gestite dal bridge

4.1 Comunicazione Bluetooth con Arduino

La comunicazione tra bridge e Arduino avviene tramite seriale Bluetooth usando il modulo **HC-05**.

Il bridge apre una porta COM locale e attende righe testuali provenienti da Arduino. Il formato previsto è il seguente:

MQ2:178

Il prefisso **MQ2**: identifica il sensore di gas, mentre il numero rappresenta il valore letto dal prototipo.

Il bridge utilizza la libreria **pyserial** per:

- aprire la connessione seriale;
- leggere le righe in arrivo;
- effettuare il parsing del valore;
- inviare i comandi di controllo della ventola.

4.2 Comunicazione MQTT con Node-RED

La comunicazione tra bridge e Node-RED avviene tramite **MQTT**, secondo il modello publish/subscribe.

Il bridge usa tre topic principali:

Topic	Direzione	Funzione
garages/G1/telemetry	Bridge -> Node-RED	Telemetria del prototipo reale G1
garages/G1/alerts	Bridge -> Node-RED	Eventi di sistema, soprattutto override manuale
garages/G1/cmd	Node-RED -> Bridge	Comandi automatici e manuali per la ventola

Il bridge pubblica telemetria e alert, mentre si sottoscrive al topic dei comandi.

5. Struttura del file `bridge.py`

Il codice del bridge e' organizzato in classi, ognuna con una responsabilita' specifica.

5.1 Classe `Config`

La classe `Config` raccoglie i parametri configurabili del bridge, ad esempio:

- porta seriale (`SERIAL_PORT`);
- baud rate (`BAUD_RATE`);
- host e porta del broker MQTT;
- topic MQTT di telemetria, alert e comandi.

Nel file `bridge.py` attuale del repository, il broker MQTT e' configurato di default come `127.0.0.1:1883`.

Questa scelta evita di disperdere i parametri nel codice e rende piu' semplice l'adattamento del bridge a contesti diversi.

5.2 Classe `BluetoothManager`

La classe `BluetoothManager` gestisce la comunicazione seriale con Arduino.

Le sue funzioni principali sono:

- aprire la seriale Bluetooth;
- leggere una riga dalla seriale;
- validare il formato `MQ2:<valore>`;
- restituire un dizionario con `gas` e `timestamp`;
- inviare ad Arduino i comandi `FAN_ON` e `FAN_OFF`.

Questa classe si occupa esclusivamente del canale Bluetooth e non contiene logica di controllo.

5.3 Classe FanController

La classe `FanController` mantiene e gestisce lo stato locale della ventola.

Le responsabilit  principali sono:

- salvare lo stato corrente della ventola (`fan_state`);
- inviare ad Arduino il comando di accensione;
- inviare ad Arduino il comando di spegnimento;
- gestire la variabile di `manual_override`;
- riportare il sistema in modalit  automatica.

La classe non decide *quando* la ventola debba attivarsi: applica semplicemente il comando ricevuto.

5.4 Classe MQTTManager

La classe `MQTTManager` gestisce il client MQTT del bridge.

Le sue funzioni sono:

- connessione al broker;
- avvio del loop MQTT;
- pubblicazione della telemetria;
- pubblicazione degli alert;
- sottoscrizione al topic dei comandi;
- ricezione e interpretazione dei messaggi provenienti da Node-RED.

Il bridge usa `paho-mqtt` e registra una callback `on_message`, che viene invocata automaticamente quando arriva un comando sul topic `garages/G1/cmd`.

5.5 Classe Bridge

La classe `Bridge` coordina l'esecuzione complessiva del programma.

Le sue responsabilit  sono:

- inizializzare i manager e il controller;
- connettere Bluetooth e MQTT;
- entrare nel ciclo principale di acquisizione;
- chiamare la procedura di pubblicazione quando arriva un nuovo valore;
- gestire l'arresto controllato.

Il bridge rappresenta quindi il livello di coordinamento, mentre le singole classi gestiscono le funzionalit  specifiche.

6. Acquisizione del dato dal sensore

6.1 Lettura dalla seriale

Durante il ciclo principale il bridge esegue ripetutamente una lettura dalla seriale Bluetooth. Il valore viene accettato solo se rispetta il formato atteso.

Se la stringa non inizia con `MQ2`: oppure non contiene un intero valido, il dato viene scartato.

Questo permette di evitare che messaggi incompleti o rumore sulla seriale entrino nella pipeline del sistema.

6.2 Validazione e parsing

Una volta ricevuta una riga valida, il bridge:

1. estrae il valore numerico del gas;
2. genera un timestamp locale in formato ISO;

3. restituisce una struttura dati del tipo:

```
{
  "gas": 178,
  "timestamp": "2026-04-17T15:20:00"
}
```

Questa struttura interna viene poi usata per costruire il payload MQTT di telemetria.

7. Pubblicazione della telemetria

Quando il bridge riceve un nuovo valore valido dal prototipo, costruisce un payload JSON e lo pubblica sul topic `garages/G1/telemetry`.

Il payload attuale contiene:

```
{
  "garage_id": "G1",
  "gas": 178,
  "timestamp": "2026-04-17T15:20:00",
  "fan_state": false
}
```

7.1 Significato dei campi

Campo	Significato
<code>garage_id</code>	Identificatore dell'unita' reale
<code>gas</code>	Valore corrente letto dal sensore MQ-2
<code>timestamp</code>	Istante in cui il dato e' stato letto dal bridge
<code>fan_state</code>	Stato attuale della ventola nel prototipo

Il payload e' volutamente minimale. La telemetria contiene solo le informazioni prodotte dal dispositivo reale e dal bridge.

8. Pubblicazione degli alert

Oltre alla telemetria, il bridge puo' pubblicare eventi di sistema sul topic `garages/G1/alerts`.

Questi eventi vengono usati soprattutto per notificare azioni di **manual override**, ad esempio:

- accensione forzata della ventola;
- spegnimento forzato;
- ritorno alla modalita' automatica.

Un esempio di payload alert e' il seguente:

```
{
  "garage_id": "G1",
  "event": "manual_override",
  "command": "FAN_ON",
  "timestamp": "2026-04-17T15:25:00"
}
```

Node-RED puo' usare questi eventi per logging, visualizzazione in dashboard e notifiche.

9. Ricezione dei comandi da Node-RED

Il bridge riceve i comandi tramite il topic `garages/G1/cmd`.

Sono supportate due famiglie di comandi.

9.1 Comandi automatici JSON

Node-RED invia un payload JSON quando il sistema e' in modalita' automatica. La struttura tipica e' questa:

```
{
  "mode": "STD",
  "anomaly": true,
  "predicted_crossing": false
}
```

Il bridge interpreta il comando in questo modo:

- se `anomaly` e' `true`, accende la ventola;
- se `predicted_crossing` e' `true`, accende la ventola;
- altrimenti spegne la ventola.

La decisione quindi non viene presa dal bridge: il bridge esegue la decisione gia' calcolata da Node-RED.

9.2 Comandi manuali

Il bridge supporta anche comandi manuali testuali:

- `FAN_ON`
- `FAN_OFF`
- `AUTO`

In presenza di `FAN_ON` o `FAN_OFF`, il bridge attiva il flag `manual_override` e forza lo stato della ventola. In presenza di `AUTO`, il flag viene disattivato e il sistema torna a seguire i comandi automatici provenienti da Node-RED.

Questo meccanismo consente di distinguere chiaramente controllo manuale e controllo automatico.

10. Gestione della ventola

Il controllo fisico della ventola avviene tramite Arduino, ma il bridge gestisce la traduzione tra comando logico e comando seriale.

Quando la ventola deve essere accesa, il bridge invia sulla seriale:

`FAN_ON`

Quando la ventola deve essere spenta, il bridge invia:

`FAN_OFF`

Allo stesso tempo aggiorna la variabile locale `fan_state`, che viene poi inclusa nella telemetria successiva.

Questo permette alla dashboard e a Node-RED di conoscere non solo il valore del gas, ma anche lo stato attuale dell'attuatore fisico.

11. Configurazione MQTT e avvio operativo

Il bridge si collega a un broker MQTT configurato tramite `MQTT_BROKER` e `MQTT_PORT`.

Nel progetto OffGas il bridge viene eseguito **localmente** sulla macchina collegata al modulo HC-05, mentre Node-RED e Mosquitto possono essere eseguiti localmente oppure in Docker.

Per questo motivo il valore dell'host MQTT deve essere coerente con il tipo di avvio scelto.

Nel codice attuale del progetto, il valore predefinito di `MQTT_BROKER` nel file `bridge.py` e' `127.0.0.1`, che puo' essere equivalentemente sostituito con `localhost` quando il bridge viene eseguito sulla stessa macchina host che espone Mosquitto sulla porta 1883.

In pratica, per il setup documentato di OffGas:

- `127.0.0.1` e' il valore consigliato e usato nel file `bridge.py` del repository;
- `localhost` e' un'alternativa valida nello stesso scenario;
- non viene piu' usato `host.docker.internal` come riferimento principale per la configurazione del bridge.

Il ciclo di avvio del bridge e' composto da questi passaggi:

1. apertura della seriale Bluetooth;
2. connessione al broker MQTT;
3. sottoscrizione al topic dei comandi;
4. avvio del ciclo continuo di lettura e pubblicazione.

12. Cosa il bridge non fa

Per capire correttamente il ruolo del bridge e' utile chiarire anche cosa **non** fa.

Il bridge non:

- calcola la soglia di sicurezza;
- legge o interpreta i dataset dei garage simulati;
- costruisce lo stato globale della dashboard;
- esegue la prediction del gas;
- decide autonomamente se una condizione sia warning o critical.

Tutte queste responsabilita' appartengono a **Node-RED**.

Questa separazione rende piu' chiara l'architettura del progetto e riduce le ridondanze tra livello edge e livello cloud.

13. Vantaggi della struttura attuale

L'organizzazione attuale del bridge presenta diversi vantaggi.

Il primo vantaggio e' la **semplicita'**. Il bridge fa poche cose, ma le fa in modo chiaro e focalizzato.

Il secondo vantaggio e' la **manutenibilita'**. Modifiche alla logica di controllo non richiedono modifiche al bridge, ma solo al flow Node-RED.

Il terzo vantaggio e' la **chiarezza architetturale**. Il bridge e' il gateway hardware, Node-RED e' il livello di orchestrazione, la dashboard e' il livello di visualizzazione.

Il quarto vantaggio e' la **portabilita'**. Il bridge puo' essere eseguito localmente sulla macchina che ospita l'HC-05, mentre il resto dell'infrastruttura puo' essere spostato in Docker senza cambiare il suo ruolo.

14. Riepilogo finale

Il bridge OffGas e' un componente edge che mette in comunicazione il prototipo reale e l'infrastruttura software del progetto.

In forma sintetica, il bridge:

- acquisisce il dato del sensore via Bluetooth;
- valida e converte il dato in una telemetria leggibile;
- pubblica la telemetria su MQTT;

- riceve comandi automatici e manuali da Node-RED;
- controlla la ventola tramite Arduino;
- pubblica gli alert di sistema legati all'override manuale.

In questa architettura il bridge non e' il luogo in cui si decide la logica del sistema, ma il componente che rende possibile il collegamento affidabile tra hardware reale e livello software di controllo.